
Date: 10/04/2026

Duration: 3 hours

FINAL ASSESSMENT

II YEAR – SEMESTER 4

CSE23AE204 - PCP III

(B.Tech. E01,E02,E03,E05,E06)

SET F

Question 1: Problem Statement

You are developing a **payment processing module** for an online platform.

The system processes **one payment request at a time**, provided as a **space-separated string**.

Each payment is handled by a specific payment mode, and each mode follows its own processing rules.

The system must:

- Identify the payment type from input
- Process the payment based on payment mode
- Deduct applicable charges
- Return the final amount that gets debited
- Use **abstraction and polymorphism** to support multiple payment types

Payment Processing Rules

UPI Payment

- A processing charge is deducted based on a percentage of the amount.
- The remaining amount is the final debited amount.

Card Payment

- A fixed service charge is deducted.
- The remaining amount is the final debited amount.

Net Banking Payment

- No processing charge is applied.
- The full amount is debited.

Example 1

Input

{ "payment": "UPI Ravi 1000 2" }

Output

980.0

Explanation

A percentage-based processing charge is deducted from the original amount, and the remaining amount is debited.

Question 2: Problem Statement

You are given an integer array nums.

Your task is to determine whether it is possible to divide the array into **two subsets** such that:

- The sum of elements in both subsets is equal

Return:

- 1 if such a partition exists
- 0 otherwise

Example 1

Input: nums = [1, 5, 11, 5]

Output: 1

Explanation:

The array can be split into [1,5,5] and [11], bot having sum = 11

Question 3: Problem Statement

An array of non-negative integers represents the **maximum jump length** that can be made from each index.

Starting at the **first index**, determine whether it is possible to reach the **last index** of the array.

From position i, a jump can reach any position in the range:

i + 1 to i + nums[i]

The task is to determine if reaching the final index is possible.

Example 1

Input: {"nums": [2,3,1,1,4]}

Output: true

Explanation Possible jumps:

index 0 -> jump 2 -> reach index 2

index 2 -> jump 1 -> reach index 3

index 3 -> jump 1 -> reach index 4

Last index is reachable.

Question 4: Problem Statement

A list of intervals is provided where each interval represents a start and end time.

Some intervals may overlap with each other.

The task is to **merge all overlapping intervals** and return the resulting set of merged intervals.

Two intervals overlap if:

current_start ≤ previous_end

When overlapping occurs, the merged interval becomes:

[start_of_first_interval, max(end_of_both_intervals)]

Example 1

Input: {"intervals": [[1,3],[2,6],[8,10],[15,18]]}

Output: [[1,6],[8,10],[15,18]]

Explanation

Intervals [1,3] and [2,6] overlap.

Merged interval:

[1,6]

Remaining intervals do not overlap.

Final result:

[[1,6],[8,10],[15,18]]

Question 5: Problem Statement

You are given a JSON object:

{ "arr": [arrival_times], "dep": [departure_times] }

- arr[i] represents the arrival time of the ith train.
- dep[i] represents the departure time of the ith train.
- Arrival and departure times are given in 24-hour format (integer, e.g., 900 for 9:00 AM).

Rules:

- A platform cannot be shared if one train's arrival time is less than or equal to another train's departure time.
- If arrival time equals departure time, a new platform is required.

Your task is to determine the **minimum number of platforms required** so that no train waits.

Example 1

Input

{ "arr": [900, 940, 950, 1100, 1500, 1800],
"dep": [910, 1200, 1120, 1130, 1900, 2000]
}

Output: 3

Explanation

After sorting:

Arrival → 900, 940, 950, 1100, 1500, 1800

Departure → 910, 1120, 1130, 1200, 1900, 2000

Overlap maximum occurs when:

- 900 arrives → 1 platform
- 940 arrives before 910 departs → 2 platforms
- 950 arrives before 910 departs → 3 platforms

Maximum platforms required = 3